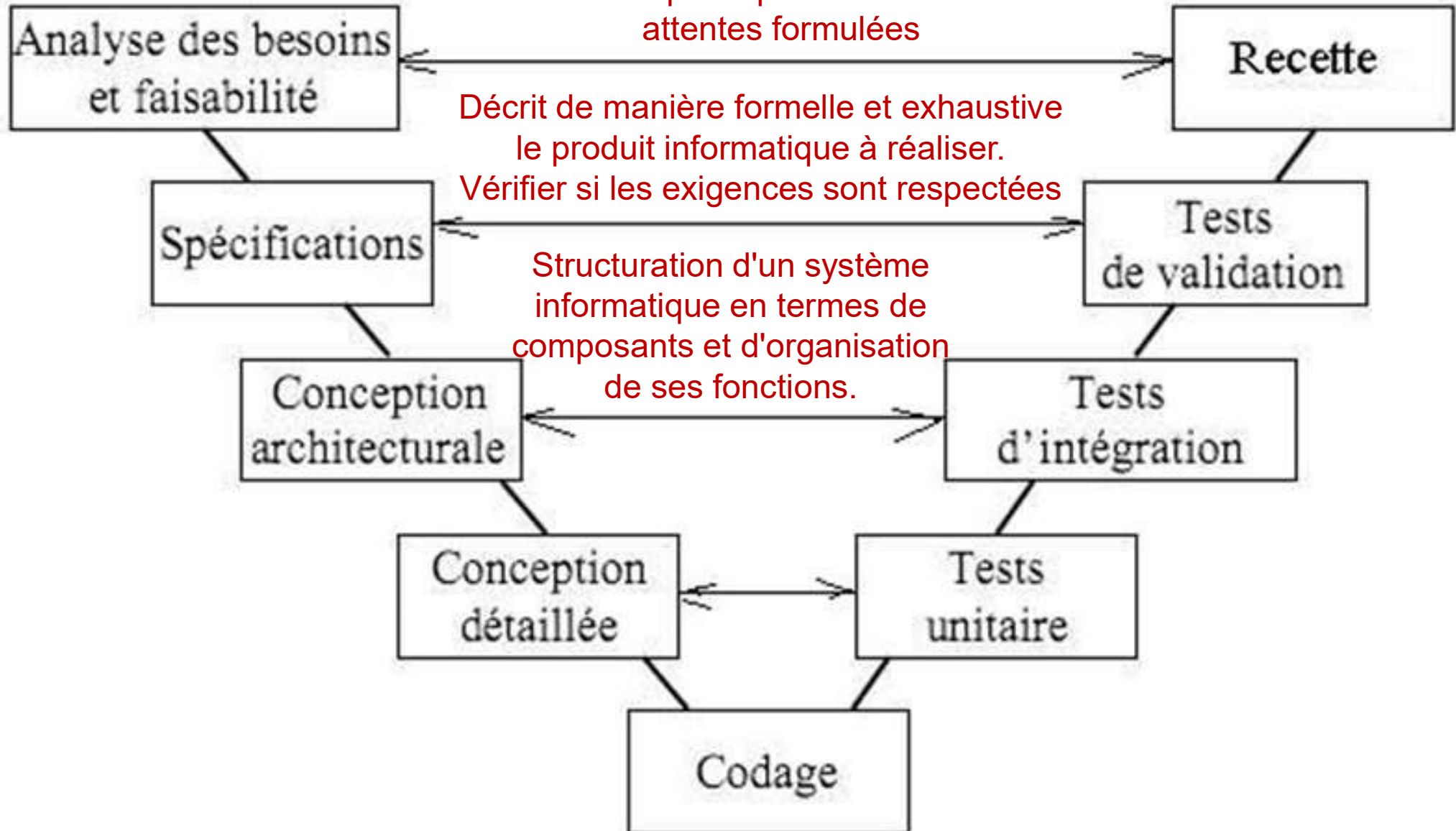


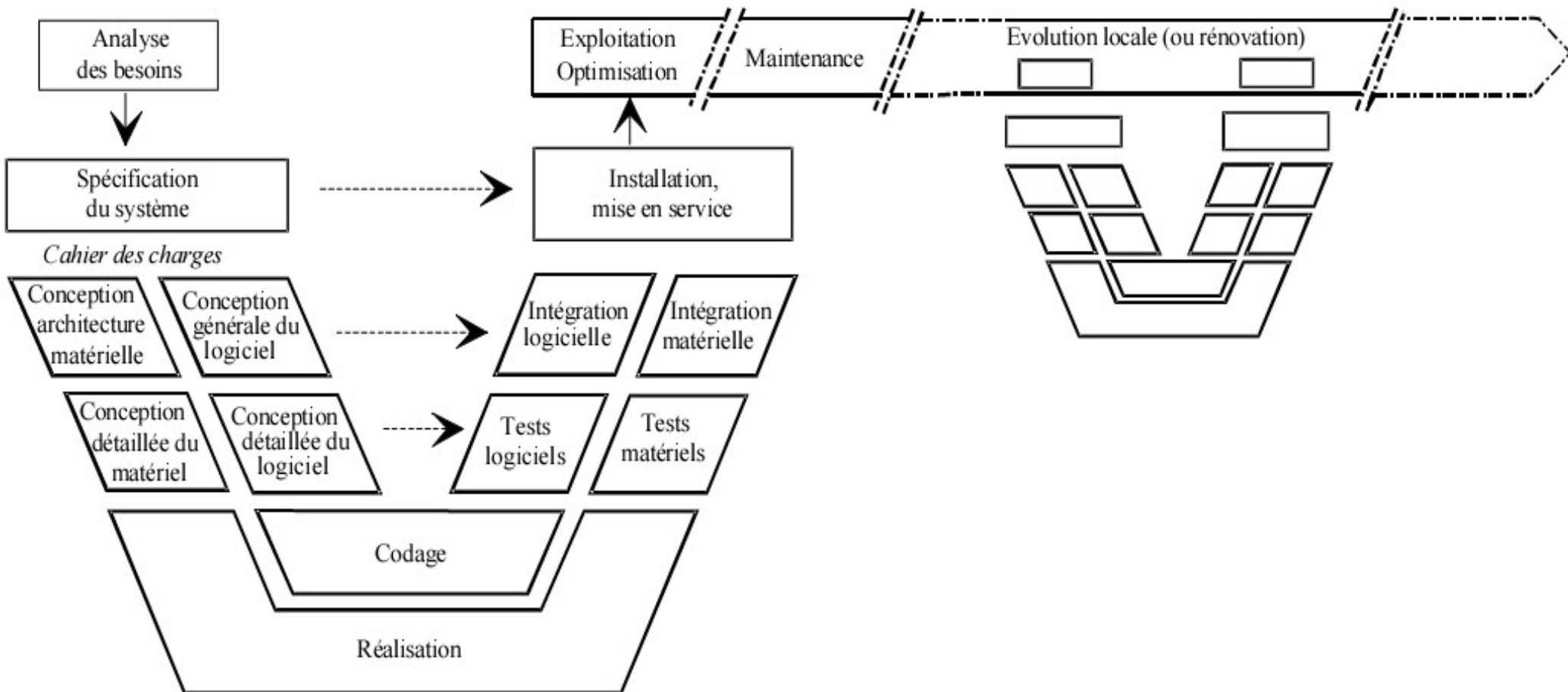
Gestion de projet

Cycle en V

Les différents acteurs du projet se rencontrent afin de vérifier que le produit est conforme aux attentes formulées



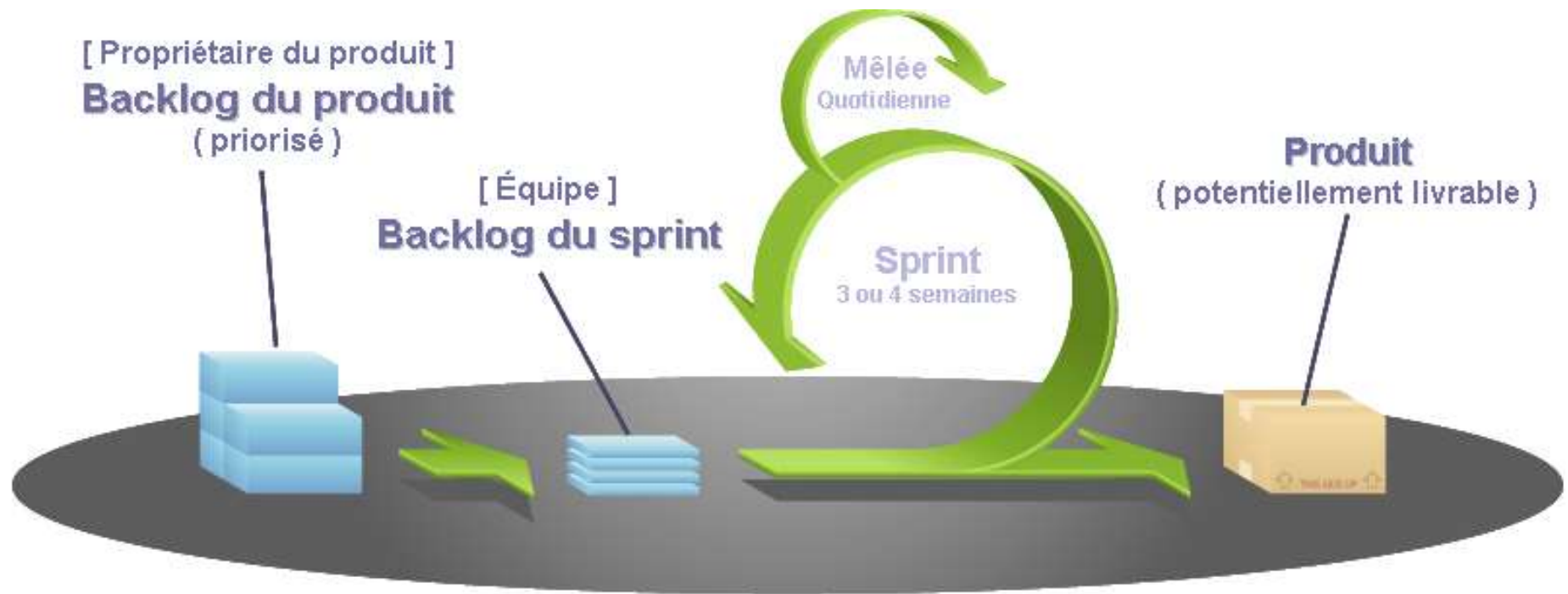
Modèle des activités de développement d'un système informatique



Gestion de projet

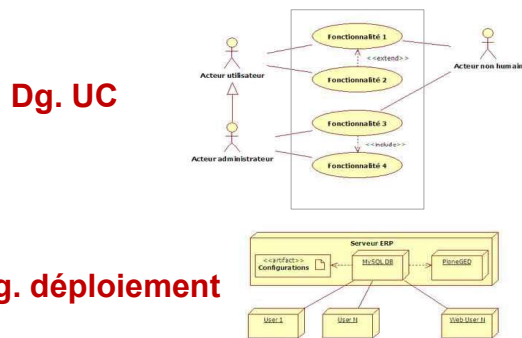
Méthode Agile

- ❑ Planification adaptative
- ❑ Développement évolutif
- ❑ Livraison précoce
- ❑ Amélioration continue
- ❑ Encouragent des réponses flexibles au changement

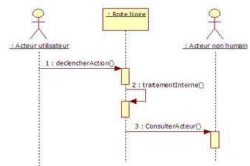
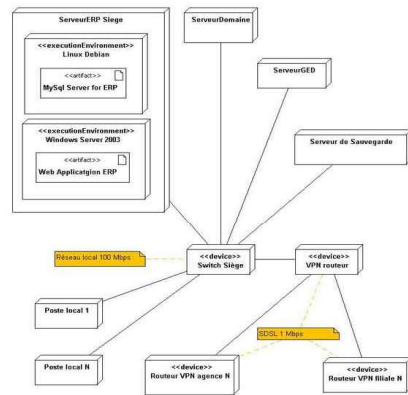


COPYRIGHT © 2005. MOUNTAIN GOAT SOFTWARE

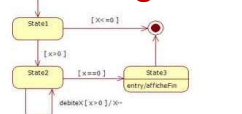
1 Expression du besoin et spécifications



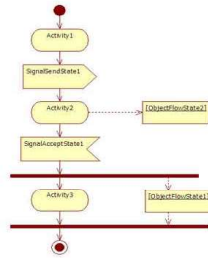
Dg. déploiement



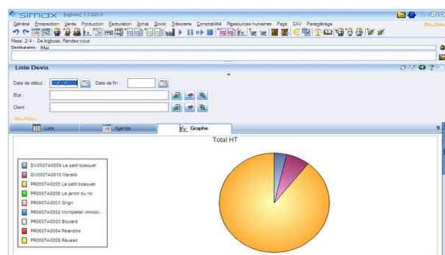
Dg. états



Dg. séquences



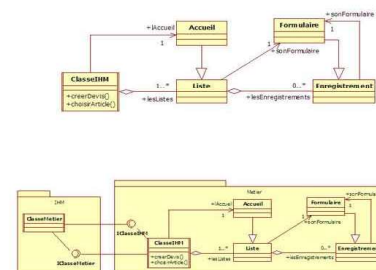
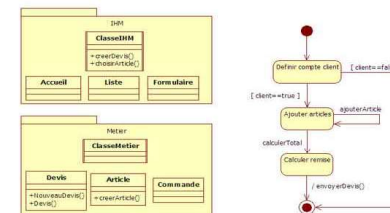
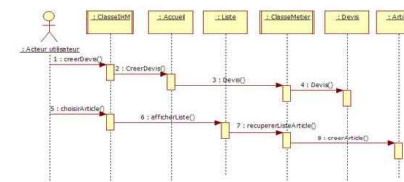
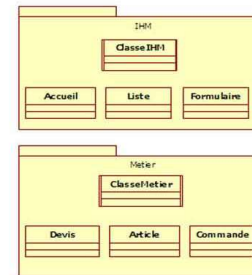
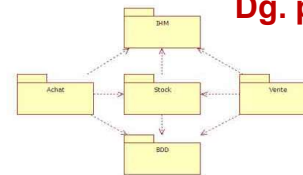
Maquettes



2 Conception détaillée



Dg. paquetages



3 Réalisation

Codage / configuration

```
//
//
// Generated by StarUML(tm) C++ Add-In
//
// @ Project : Untitled
// @ File Name : ClasselHM.h
// @ Date : 14/07/2008
// @ Author :
//
//
```

```
#if !defined(_CLASSEIHM_H)
#define _CLASSEIHM_H

#include "ClasseIHM.h"
#include "Liste.h"
#include "Accueil.h"

class ClasseIHM : public IClasseIHM {
public:
    void creerDevis();
    void choisirArticle();
    Liste *lesListes;
    Accueil *lAccueil;
};

#endif // _CLASSEIHM_H
```

```
//
//
// Generated by StarUML(tm) C++ Add-In
//
// @ Project : Untitled
// @ File Name : ClasselHM.cpp
// @ Date : 14/07/2008
// @ Author :
//
//
```

```
#include "ClasselHM.h"

void ClasselHM::creerDevis() {
}

void ClasselHM::choisirArticle() {
}
```

Capture des besoins et conception détaillée (analyse)

1 Expression du besoin et spécifications

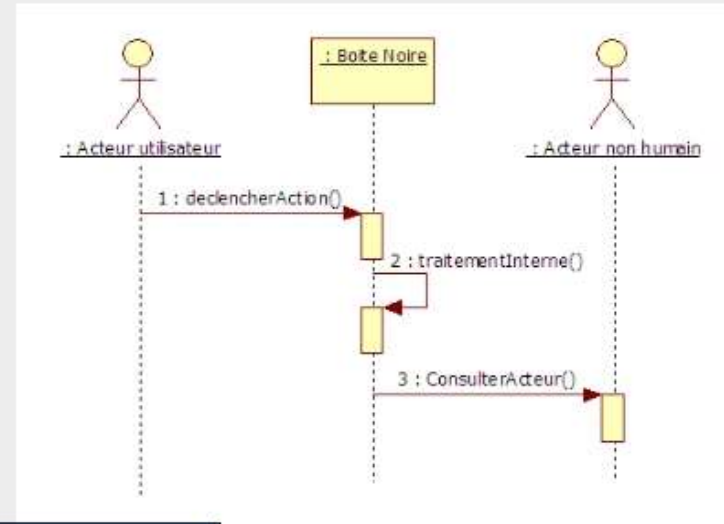
1 - Capture des besoins => Boite noire

Délimiter le système (boite noire) et les acteurs



Fonctionnalités → cas d'utilisation

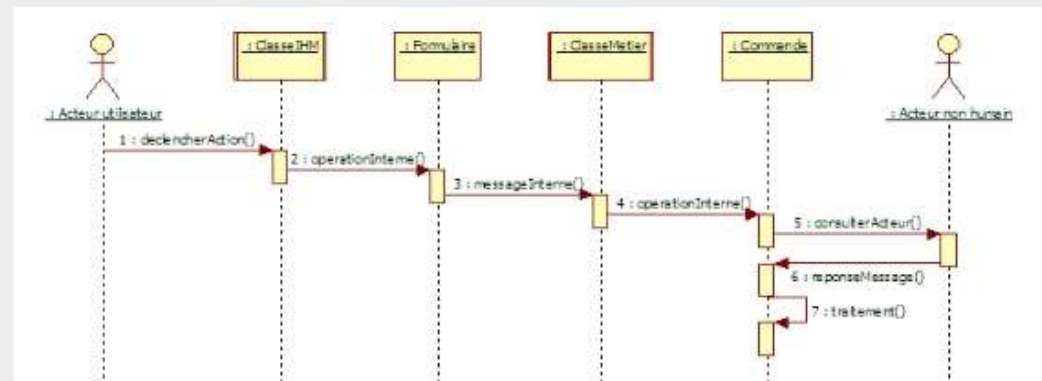
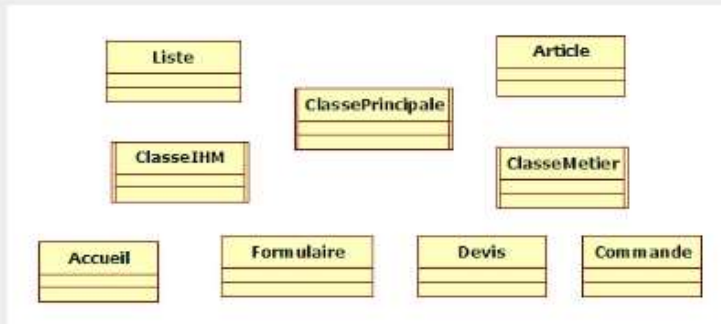
Scénarii liés aux cas d'utilisation → Séquences



2 Conception détaillée

Définir les entités → base de données et classes

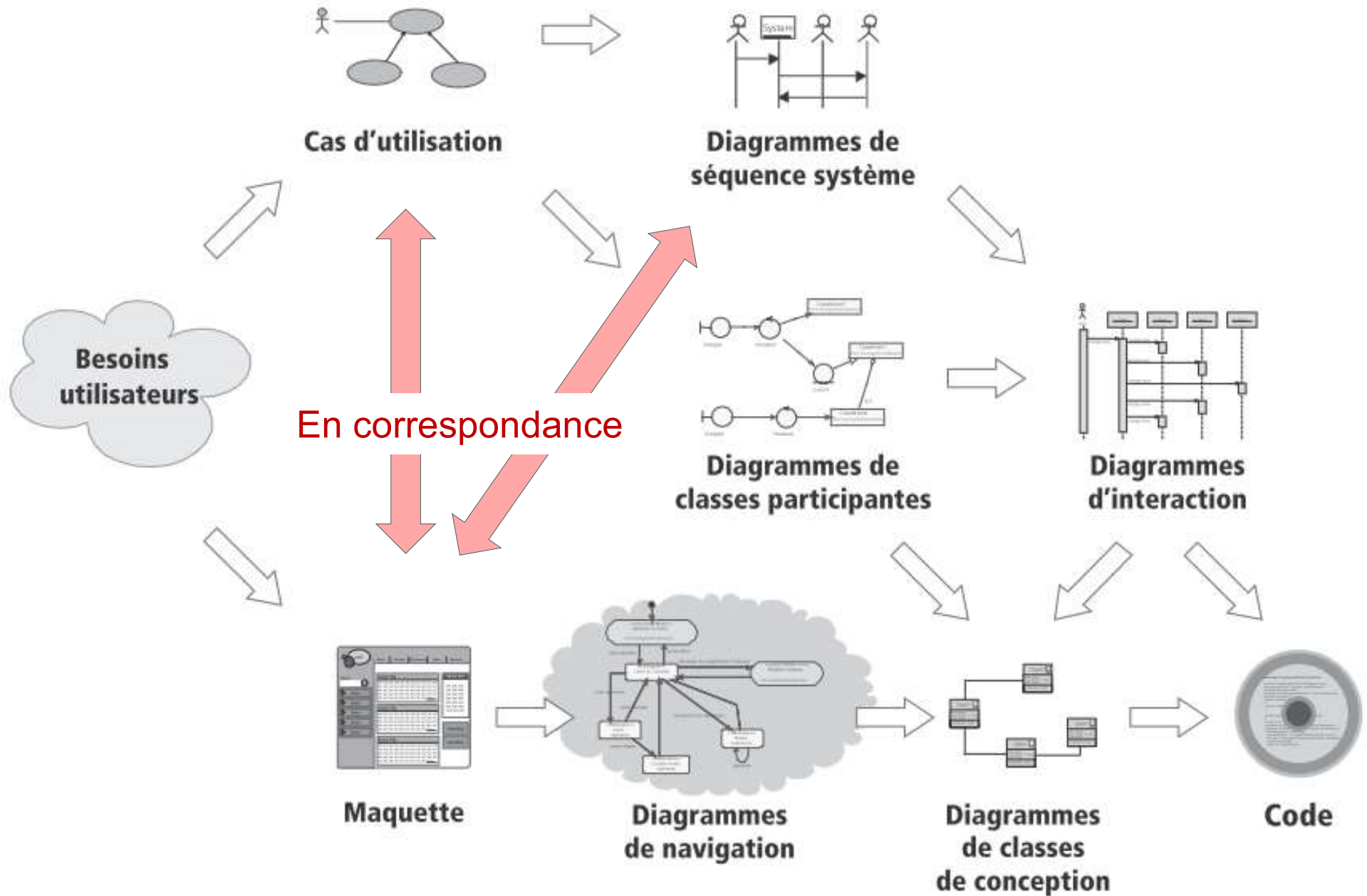
Décomposer le système en éléments simples → classes, modules



<http://www.uml-sysml.org/uml/lanalyse-et-la-conception-architecturale>

Processus de modélisation

(ex. application web)



Conception détaillée / Analyse

Objectifs :

- Identifier les classes/entités
- Concevoir la BDD
- Définir les packages et l'architecture générale
- Regrouper les classes par package et les interfaces de packages
- Affiner la définition statique et dynamique des classes
- Générer l'architecture de départ

Diagrammes UML/Sysml :

- ☐ de classe
- ☐ de package
- ☐ de séquence
- ☐ d'état
- ☐ d'activité
- ☐ d'objet

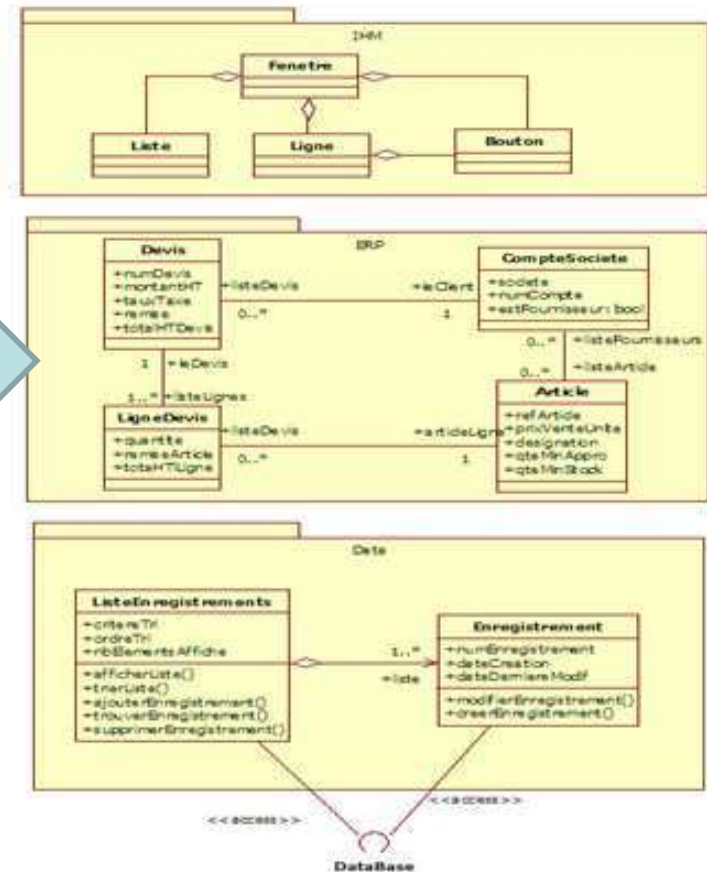
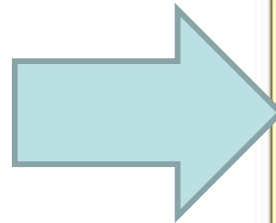
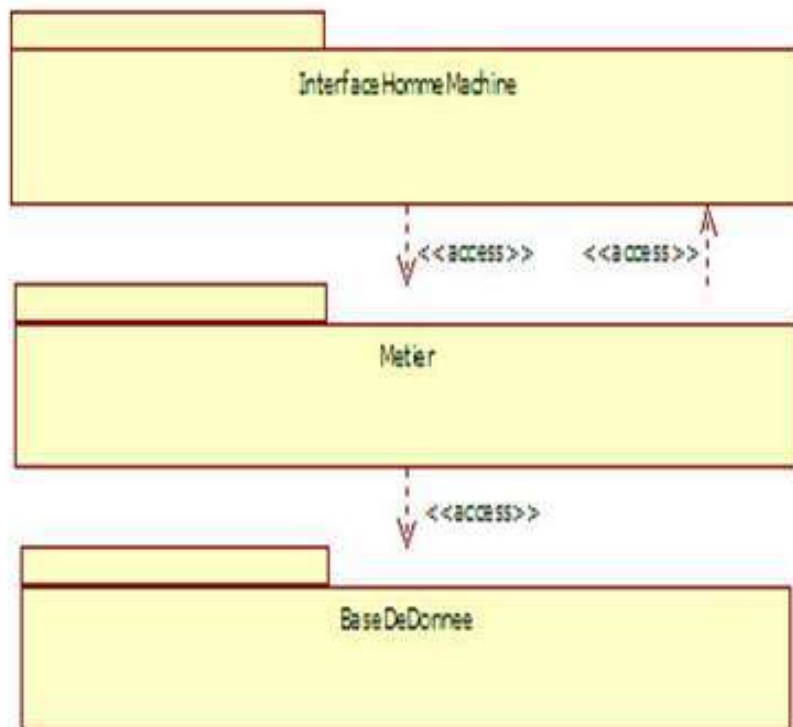
Identification des classes

(Représentation statique du système)

- Pour chaque classe déterminer son métier, sa responsabilité et ses attributs.
Si une classe présente de trop nombreuses responsabilités, il faut la décomposer.
- Séparer les classes :
 - Représentant les données qui seront dans la BD (que des attributs) voir ORM
 - Constituant la partie logiciel du projet (attributs et méthodes)
- Etablir les associations entre les classes

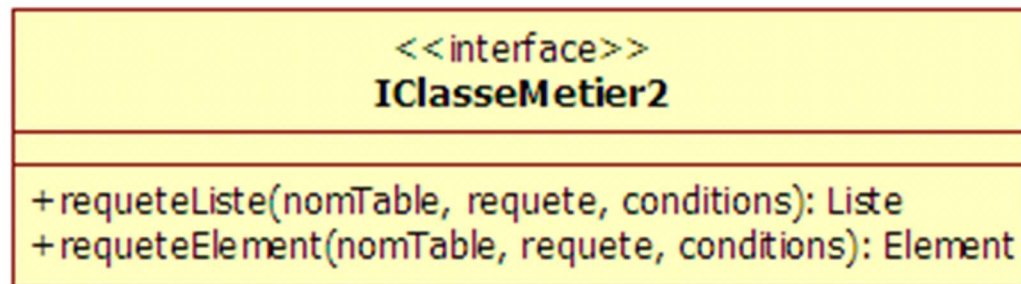
Répartition des classes en packages

- Modularité générale du système
- Regroupement en Packages des classes
- Souvent **architecture 3 tiers**
- Le package définit un espace de nom et une unité organisationnelle

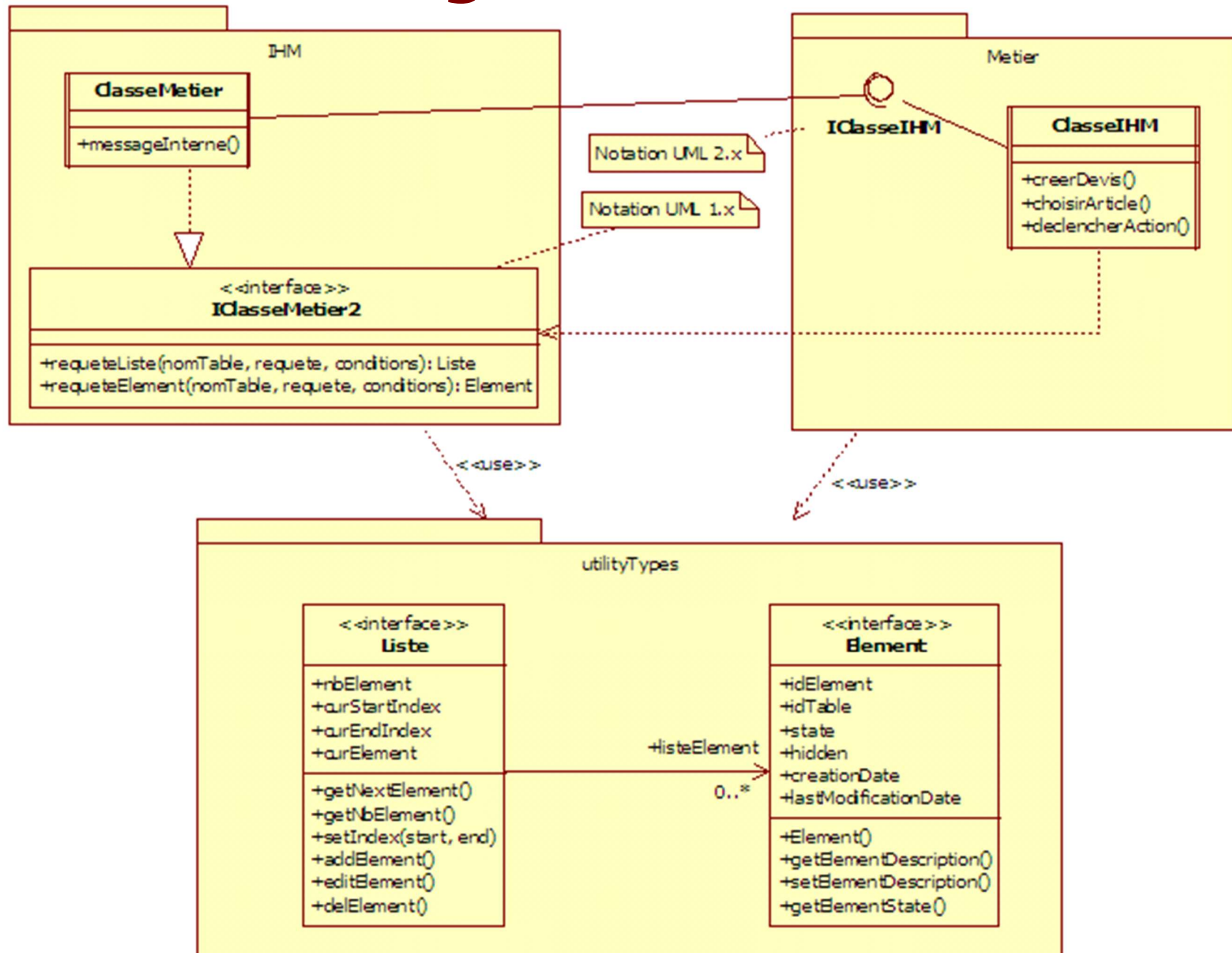


Définition des interfaces entre packages

- Les interfaces ont un rôle d'encapsulation des classes du package vis à vis du reste du système.
- Rôle majeur dans la maintenabilité et l'évolutivité du système.
- Une interface UML est une classe ne contenant que des méthodes abstraites comme en C++ ou une interface comme en Java.



Packages et interfaces



En parallèle de expression du besoin et spécifications

Préparation de la chaine de production

- Environnement de gestion de projet (**Azure DevOps**) et de modélisation (**MagicDraw**, mySQLWorkbench...)
- Environnement de compilation (Visual studio, Android Studio, QtCreator ..)
- Environnement d'exécution (VMs, Containers, Serveurs...)
- Procédures de stockage des versions (**Git**)
- Charte informatique
- Plateforme de test (tests unitaires)

Choix et tests de solutions techniques

- Se renseigner sur les solutions possibles (avantages/inconvénients)
- Si non imposé, faire un choix en comparant différentes solutions en précisant les critères
- Maîtriser les concepts et l'utilisation (appropriation)
- Réaliser des tests sur des exemples simples « Hello World »
- Réaliser des tests de faisabilité par rapport aux exigences du projet

Mise en œuvre du système existant